

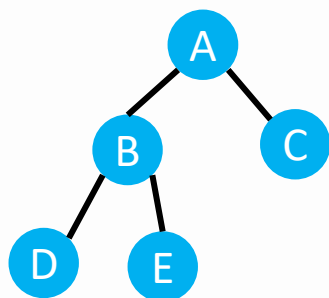
树的遍历

树的遍历方法有深度优先遍历和广度优先遍历，遍历方法和搜索类似。

```
void dfs(int u,int fa)
{
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
    }
}
```

树的DFS序

当我们对树进行dfs遍历的时候，对于每个结点，我们在**进入**和**出去**的时候都记录一下该点的编号，这样我们得到的长度为 $2N$ 的序列就是树的DFS序。



树的DFS遍历：ABDEC

树的DFS序：ABDDEEBCCA

树的DFS序的作用：我们会发现在一棵树中，以结点 i 为根结点的子树在DFS序中就是以 i 为起点， i 为终点的一段区间。比如BDDEEB就表示以B为根结点的子树，每个结点刚好都被算了两次。

树的DFS序

```
void dfs(int u,int fa)
{
    L[x]=++cnt;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
    }
    R[x]=++cnt;
}
```

DFS序的好处是他能把树上的问题转换成区间问题来求解——比如求解带修改的树上子树查询问题，我们就可以把树转换成DFS序列，然后再用单点修改，区间查询来解决。虽然树剖也可以轻松的解决，但是DFS+树状数组好写啊

苹果树

一棵苹果数，每个树杈上可以长一个苹果，也可以把这个该结点的苹果摘掉，现给定m个询问，询问以i为根结点的树杈上此时有多少个苹果。

输入：

3

1 2

1 3

3

Q 1

C 2

Q 1

输出：

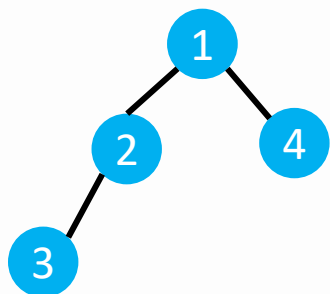
3

2

$N, M \leq 100000$

POJ 3321 苹果树

苹果树



DFS序: 1 2 3 3 2 4 4 1

根据题目要求，操作有单点增加和单点减少，询问和子树查询，对于dfs序来说，一个结点的子树的编号肯定是连续的，在dfs中就是连续的一段区间，所以子树查询可以用线段树来做，也可以用树状数组的前缀和来做。对于单点修改，只需要把两个位置都修改就可以了。

但是能不能更加简单一点呢？

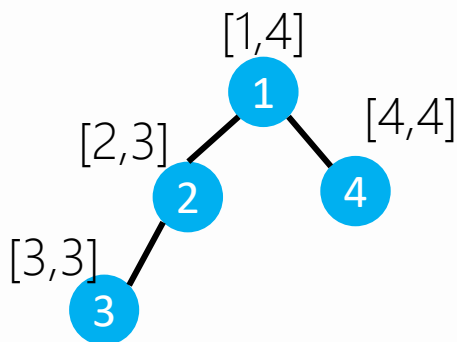
实际上我们可以直接用线段树的表示方法，比如节点1就表示真实区间 $[1, 4]$ ，节点2就表示真实区间 $[2, 3]$ ，这样长度就还是 N 。实际上就是出去的时候cnt不变。

苹果树

```
void dfs(int u,int fa)
{
    L[x]=++cnt;//进入时间
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
    }
    R[x]=cnt;//出去时间
}
```

这就是为什么进来的时候++cnt, 而出去的时候没有++cnt的原因, 因为出去的时候点的个数并没有变化, 所以cnt不会变。

那么这种做法有什么好处呢?



苹果树

第一步：建树。

第二步：dfs遍历，记录每个点表示的区间 $L[x], R[x]$ 。

第三步：初始化树状数组

第四步：树状数组修改(增加，减少)

第五步：树状数组前缀和查询。

DFS 序

子树修改，子树查询

根据dfs的思想， he 可以把树上修改问题转换成区间修改和查询问题，一颗子树的dfs序实际上就是一段连续的区间，那么对于子树修改和子树查询实际上就对应了线段树的区间修改和区间查询。

LOJ #145 DFS序2

树的DFS序

```
void dfs(int u,int fa)
{
    L[x]=++cnt;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
    }
    R[x]=++cnt;
}
```

树的深度

```
d[root]=1;
void dfs(int u,int fa)
{
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            deep[v]=deep[u]+1;
            dfs(v,u);
        }
    }
}
```

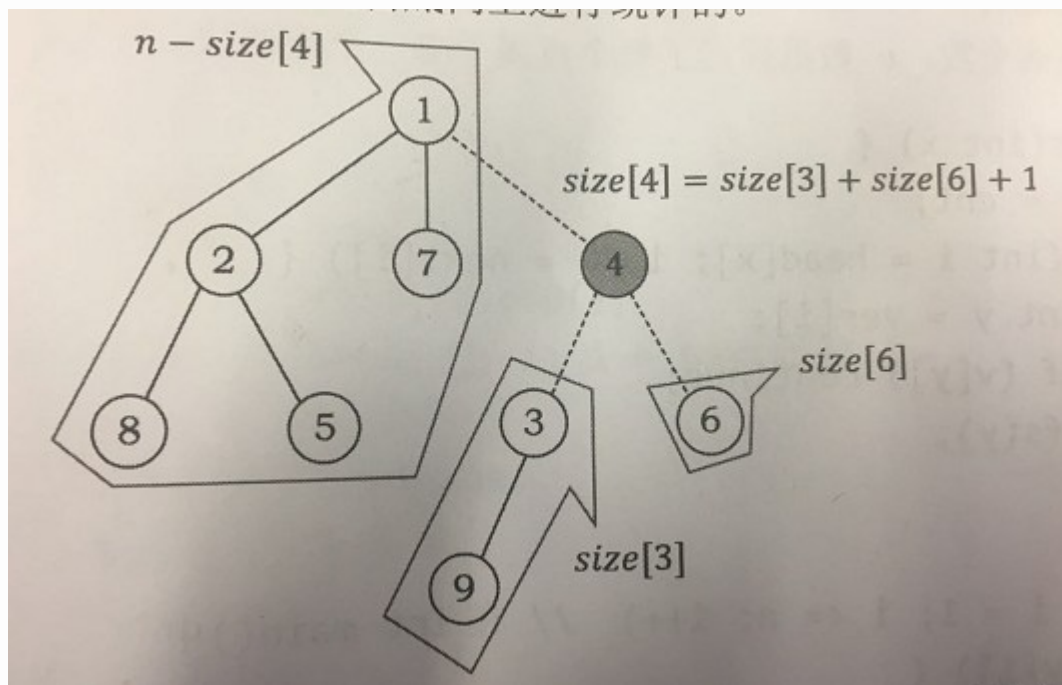
树的节点数

```
void dfs(int u,int fa)
{
    size[u]=1;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            dfs(v,u);
            size[u]+=size[v];
        }
    }
}
```

树的重心

找到一个点提根，其所有子树中最大的子树结点数量最少，这个点就是这颗树的重心，删除重心后，生成的多棵树尽可能平衡。

P0J 1655 树的重心
洛谷 P1395 会议



树的重心

```
void dfs(int u,int fa)
{
    size[u]=1;
    int Max=0;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            dfs(v,u);
            size[u]+=size[v];
            Max=max(Max,size[v]);
        }
    }
    Max=max(Max,n-size[u]);
    if(Max<ans)
    {
        ans=Max;
        pos=u;
    }
    else if(Max==ans&&pos>u)
        pos=u; //选择编号最小的
}
```

树的重心

- 1: 树的重心的每棵子树的大小一定小于等于 $n/2$
- 2: 每棵子树大小都小于等于 $n/2$ 的点一定是这棵树的重心
- 3: 树中所有点到某个点的距离和中，到重心的距离和最小，如果有两个重心，他们的距离一样。
- 4: 两棵树通过一条边相连成为一棵新树，新树的重心一定在原来两棵树的重心的路径上

树的直径

树中任意两个结点的简单路径的距离的最大值。

求解方法：

先从任意一个点跑一遍dfs，走到离当前起点最远的点（如有多个，随便选择一个），该点就是树的直径的一个端点，再从这个最远的点跑一遍dfs，离当前起点最远的点就是直径的另一个端点，他们之间的距离就是树的直径。

证明：

如果起点 u 在树的直径上，最远的点 v 一定也在直径上（否则他就不是最远的点）；如果起点 u 不在树的直径上，那么 $u-v$ 一定和树的直径相交，交点一定为 v ，否则 $u-v$ 不可能最长，其中一部分替换为直径的另一部分更长。

树的直径

给你一颗N个节点的树，求每个点到其他点的最大距离。

输入：

5

1 2

1 3

1 4

4 5

输出：

2 3 3 2 3

$N \leq 500000$

MSOJ 1345 最长链

树的直径

树的直径的性质：

- 1: 树上任意一点到树的最远距离就是该点到树的直径的其中一个端点的距离。
- 2: 如果一棵树有多条直径，这些直径交于一点，并且该点是直径的中点。
- 3: 两棵树用一条边相连，新树的直径的两端一定是原本两棵树直径四个端点中的两个。

洛谷 P1099 树网的核

洛谷 P4408 逃学的小孩

洛谷 P2195 HXY造公园

洛谷 P3761 城市

树的中心

对于一棵树，定义 $r(x)$ 为离 x 最远的点到 x 的距离，树的重心为满足 $r(x)$ 最小的点 x 。

1：所有的直径都经过树的中心

2：中心是任意一条直径的中点，对于直径上有偶数个点的情况存在两个中心。

树上倍增

树上倍增就是用来求解不带修改的树上查询问题，他的原理和RMQ一致，都是使用倍增跳跃的思想，这样就能把一步一步向上跳跃 $O(n)$ 的时间复杂度优化为 $O(\log n)$ 。

在RMQ中， $f[i][j]$ 表示以 i 点为起点，长度为 2^j 的区间 $[i, i+2^j-1]$ ，那么该区间可以由 $f[i][j-1]$ 和 $f[i+2^{j-1}-1][j-1]$ 合并而来。

同样，在树上倍增中， $f[i][j]$ 表示 i 点向上跳跃 2^j 步的位置，那么 $f[i][j]$ 可以由 $f[f[i][j-1]][i-1]$ 得到，也就是说深度越深的点可以由深度较浅的点转移过来。这样我们只需要预处理 $i * \log(\text{deep})$ 的数组就可以存储所有点向上跳跃 2^j 的步数的位置。

树上倍增

```
void dfs(int x)
{
    book[x]=true;
    for(int i=1;(1<<i)<=deep[x];i++)// 求出该点往上 $2^k$ 的点的位置。
    {
        fa[x][i]=fa[fa[x][i-1]][i-1];
    }
    for(int i=head[x];i;i=edge[i].next)// 深度优先遍历
    {
        int u=edge[i].to;
        if(!book[u])
        {
            fa[u][0]=x;// x是u的父亲
            deep[u]=deep[x]+1;// u的深度是x的深度+1
            dfs(u);
        }
    }
}
```

树上倍增

预处理好之后就是查询任意两点的最近公共祖先，最大的问题就是很有可能最近公共祖先所在的深度和两点的深度之差不是刚好等于 2^i 。

但是我们知道对于任意一个十进制数都可以拆分成一个唯一的二进制数，那么对于任意深度的点，都可以通过一系列 2^i 最终到达深度比他浅的点，只是需要跳几次而已，并且跳跃次数不会多于 $\log n$ 。所以我们只需要判断两个点向上跳知否可以跳到同一个点就可以了，为了方便，我们先把这两个点放在同一层，每次跳跃相同的高度，因为一旦跳到根结点以及根结点之上，那么祖先肯定是相等，如果不相等，说明肯定可以跳上去，并且可以先同时跳到该位置，再继续跳。

注意最开始要特判两个点是否相等。

树上倍增

```
for(int i=0;(1<<i)<=t;i++)//先调整到同一高度，即把u向上调整t个高度
{
    if((1<<i)&t)//二进制数，如果该位有1就跳
    {
        u=fa[u][i];//一定要更新u。不然全错
    }
}
if(u==v)//如果u,v在一条链上，直接输出上面那个店
return u;
for(int i=20;i>=0;i--)//i可以等于0，表示其父亲结点
{
    if(fa[u][i]!=fa[v][i])
    {
        u=fa[u][i];//更新
        v=fa[v][i];
    }
}
return fa[u][0];//u和v是最后一个不相等的结点其父亲结点是相等的，所
```

树上倍增

倍增求解两点之间的距离：

对于一棵树上的任意两点，直接求解他们的距离比较麻烦，我们可以把这条链分成两段，分别求解这两个点到他们的lca的距离。

如果边权相同，我们可以直接变跳跃边累加，但是如果边权不相等呢？跳跃的次数并不等于跳跃的距离。

我们仍然可以考虑前缀和的思想，记录每一个点到根结点的距离 $d[i]$ ，那么对于两点 $[x, y]$ 之间的距离就变成了 $d[x] + d[y] - 2 * d[lca(x, y)]$ 。

树上差分

明白了树上倍增的原理就知道，先预处理再查询，所以是不支持修改的，只要任意一个点发生变化，这条链上的所有点的祖先都会发生变化，那么修改的时间复杂度是比较高的。

但是对于某一种修改：

每次修改一条链，最后只有一次查询(少量)

我们可以参考种树问题的差分思路来求解。

也就是说如果种树的那一条是一棵树，每次挖去的是两个点之间的简单路径的树，最后问还剩下多少棵树，应该怎么做？

树上差分

题目描述

展开

松鼠的新家是一棵树，前几天刚刚装修了新家，新家有 n 个房间，并且有 $n-1$ 根树枝连接，每个房间都可以相互到达，且两个房间之间的路线都是唯一的。天哪，他居然真的住在“树”上。

松鼠想邀请小熊维尼前来参观，并且还指定一份参观指南，他希望维尼能够按照他的指南顺序，先去 a_1 ，再去 a_2 ，……，最后到 a_n ，去参观新家。可是这样会导致维尼重复走很多房间，懒惰的维尼不停地推辞。可是松鼠告诉他，每走到一个房间，他就可以从房间拿一块糖果吃。

维尼是个馋家伙，立马就答应了。现在松鼠希望知道为了保证维尼有糖果吃，他需要在每一个房间各放至少多少个糖果。

因为松鼠参观指南上的最后一个房间 a_n 是餐厅，餐厅里他准备了丰盛的大餐，所以当维尼在参观的最后到达餐厅时就不需要再拿糖果吃了。

输入格式

第一行一个整数 n ，表示房间个数
第二行 n 个整数，依次描述 a_1-a_n

接下来 $n-1$ 行，每行两个整数 x, y ，表示标号 x 和 y 的两个房间之间有树枝相连。

输出格式

一共 n 行，第 i 行输出标号为 i 的房间至少需要放多少个糖果，才能让维尼有糖果吃。

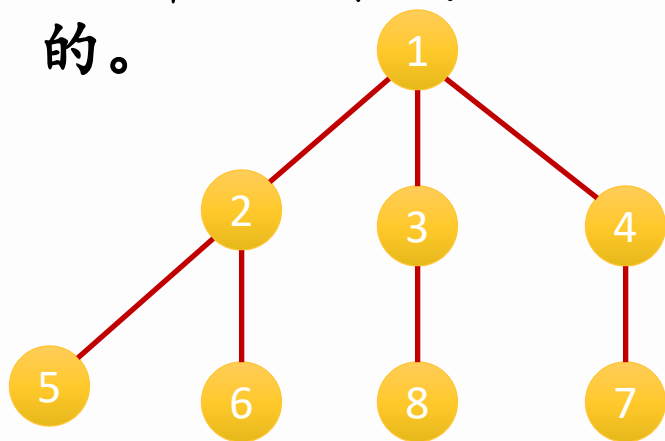
树上差分

按照前面差分的思想，我们要把一条链的修改转换成单点修改，把树上一个点的权值转换成前缀权值。

直接把任意两个点之间的简单路径换成前缀会比较麻烦，所以我们把一条路径看做两部分。

比如 $a \rightarrow b$, $\text{lca}(a, b) = c$, 那么我们可以看成两条链 $a \rightarrow c$, $b \rightarrow c$ 。

那么如果我们按照这种方式进行差分，明显是有问题的。



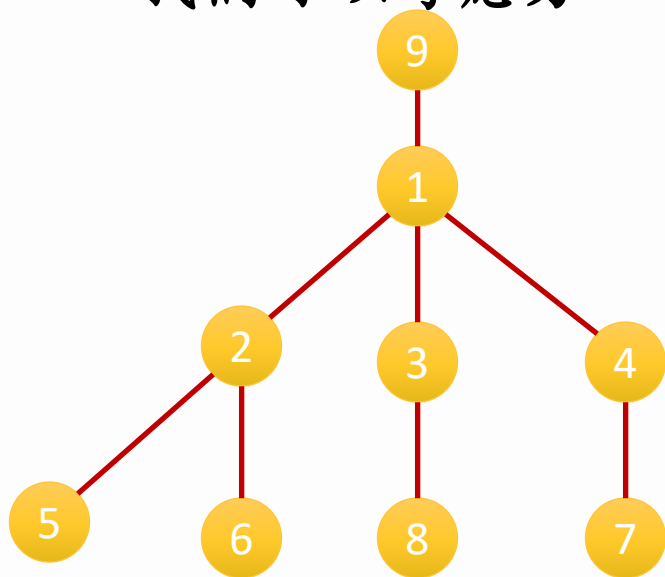
比如修改链 $2 \rightarrow 4$, 那么就是 $a[1]++$, $a[5]--$, $a[1]++$, $a[7]--$, 这样修改过的链就是正确的, 但是对于其他的链比如 $1 \rightarrow 8$ 就错误了, 所以我们不能这样差分。

树上差分

差分的方式有很多种，只要差分之后能保证一个点的权值等于差分数组部分之和就可以了，并且是有规律的。

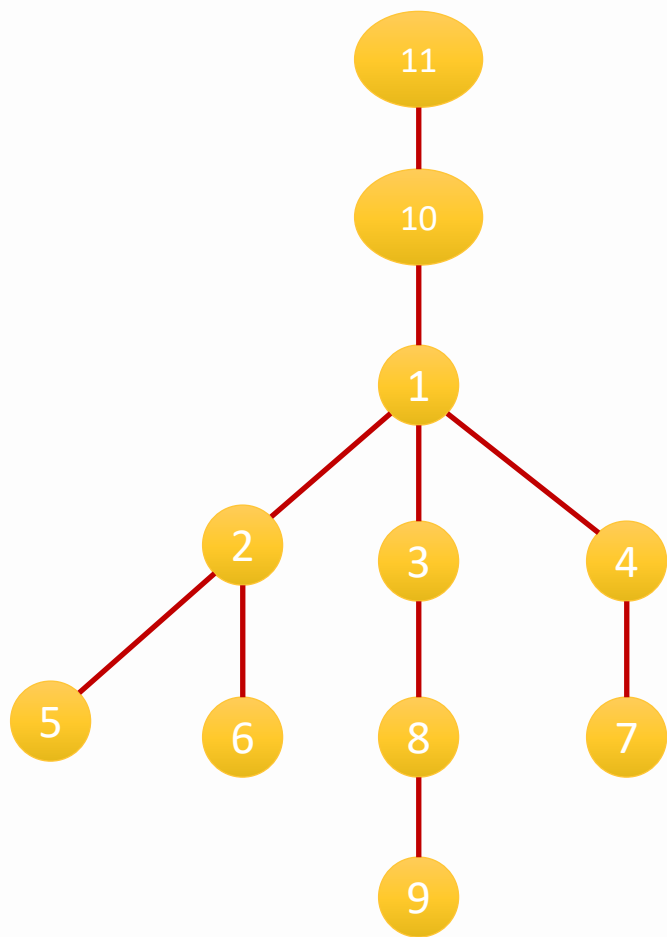
既然从上面往下面差分有一点问题，那么我们可以尝试一下从下面往上面差分。

我们可以考虑另一种差分方式：把子树之和当做点权



比如修改链 $2 \rightarrow 4$ ，那么就是 $a[2]++$, $a[4]++$ ，我们再考虑他们的最近公共祖先，该位置的子树 $+2$ ，但是实际只加了1，所以 $a[1]--$ ，但是1的祖先9并没有改变，所以 $a[9]--$ ，我们再看其他的，3的子树并没有改变，2,4的儿子也没有改变

树上差分



因为是差分+前缀和的思想，所以对于每一个点，我们可以看做子树之和。假设我修改一条链 $2 \rightarrow 4$ ，那么为了让2的子树不受影响，并且最终2的答案只减少1，那么我们只需要修改 $cf[2]$ ，4也同理，那么不会影响到2,4的子树和其他链的，但是祖先1的子树之和-2，实际只减少了1，那么 $cf[1]+1$ 才能保持不变。我们再考虑整棵子树，1所在树总的数量-1,1上面的点统计答案的时候也会减少，所以1的祖先 $cf[10]+1$. 这样该减的就减掉了，其他的都不受影响。

树上差分

第一步：树上倍增预处理祖先

第二步：根据链的修改处理差分数组。

```
for(int i=2;i<=n;i++)  
{  
    int pa=lca(a[i-1],a[i]);  
    cf[a[i-1]]++;  
    cf[a[i]]++;  
    cf[pa]--;  
    cf[fa[pa][0]]--;  
}
```

第三步：dfs统计子树答案。



练习题

洛谷 P3128 最大流
洛谷 P3268 松鼠的新家
洛谷 P2680 运输计划
洛谷 P1600 天天爱跑步



树剖求LCA

对于树上路径的查询，为了更高效的查询他们的最近公共祖先，我们采用的是树上倍增的思想，但是如果我們仔细思考，我们会发现，还是有点慢。首先，每次向上跳的时候都是枚举的 2^{20} 左右，实际上一棵树很难有这么深的深度；其次，就算一棵树的深度很大，但是任选的两个点的深度就未必有这么大，所以我们还可以找到更快的向上跳的办法——基于树链剖分思想的轻重链跳跃。

在树剖中，我们可以直接从一个点跳跃到重链的顶点，如果是轻链则跳跃到轻链的父亲节点，但是轻重链剖分之重，可以证明一棵深度为 n 的树，最多跳跃 $\log n$ 次就可以跳跃到根结点。

所以我们可以用树剖来做，当然只需要前面的剖分就可以了，也就是两个dfs+一个跳跃。

树剖求LCA

```
void dfs1(int u)//第一次求fa数组, size和重儿子
{
    size[u]=1;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa[u])
        {
            fa[v]=u;
            deep[v]=deep[u]+1;
            dfs1(v);
            size[u]+=size[v];
            if(!son[u]||size[v]>size[son[u]])
                son[u]=v;
        }
    }
}
```


树剖求 LCA

```
void dfs2(int u,int tp)//第二次轻重链剖分
{
    top[u]=tp;
    if(son[u])
        dfs2(son[u],tp);
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa[u]&&v!=son[u])
            dfs2(v,v);
    }
}
```

树剖求 LCA

```
int lca(int u,int v)
{
    while(top[u]!=top[v])
    {
        if(deep[top[u]]<deep[top[v]])
            v=fa[top[v]]; //始终让深的往浅的跳
        else
            u=fa[top[u]];
    }
    return deep[u]<deep[v]?u:v;
    //注意停止条件，在一条链上就停止而不是到同一个点在停止
}
```